

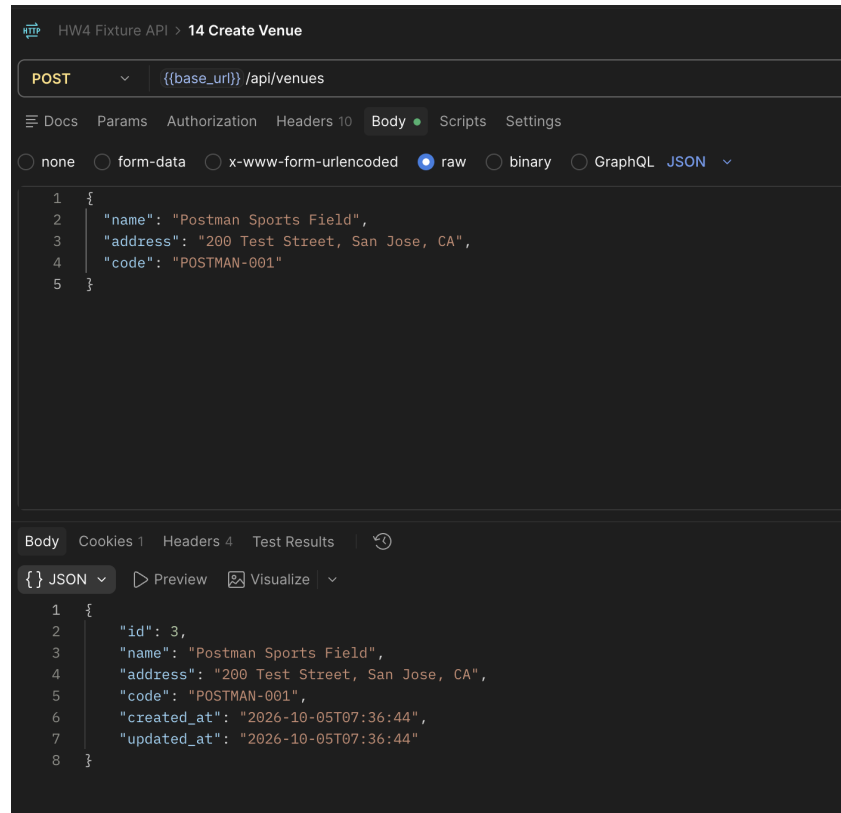
Homework 5 Report

Section 0 - Personal Configuration and Domain

Value	Definition
SID4	2383
PORT_BASE	8583
PREFIX	s2383
SEED	2383
VERIFY_SEED	262383
DOMAIN_ID	7
Hardware	MacBook Pro, Apple M4 (10-core CPU), 16 GB RAM
Python version	3.11.9
Tagged commit hash	

Part 1

- I. **Database** - Related to the next sections so relevant code and outputs shown there directly.
- II. **API Implementation**
 1. **Create Venue**



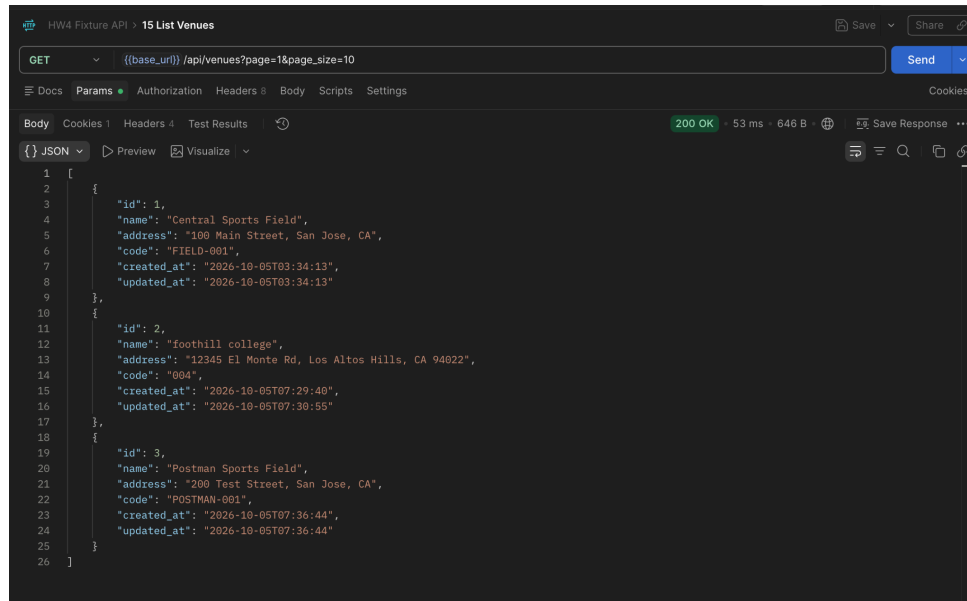
```
@app.post("/api/venues", status_code=201, response_model=VenueOut)
def create_venue(
    request_data: VenueCreate, # FastAPI reads the JSON request body
    db: DbSession = Depends(get_db), # database session using the ex
    user: User = Depends(get_current_user), # checks that the user is
):
    venue = Venue(
        name=request_data.name,
        address=request_data.address,
        code=request_data.code,
    )

    try:
        db.add(venue)
        db.commit()
        db.refresh(venue)
    except IntegrityError:
        db.rollback()
        raise HTTPException(
            status_code=409,
            detail="Venue code already exists",
        )

    return venue_to_dict(venue)
```

Ananya Yallapragada
019142383

2. List Venues



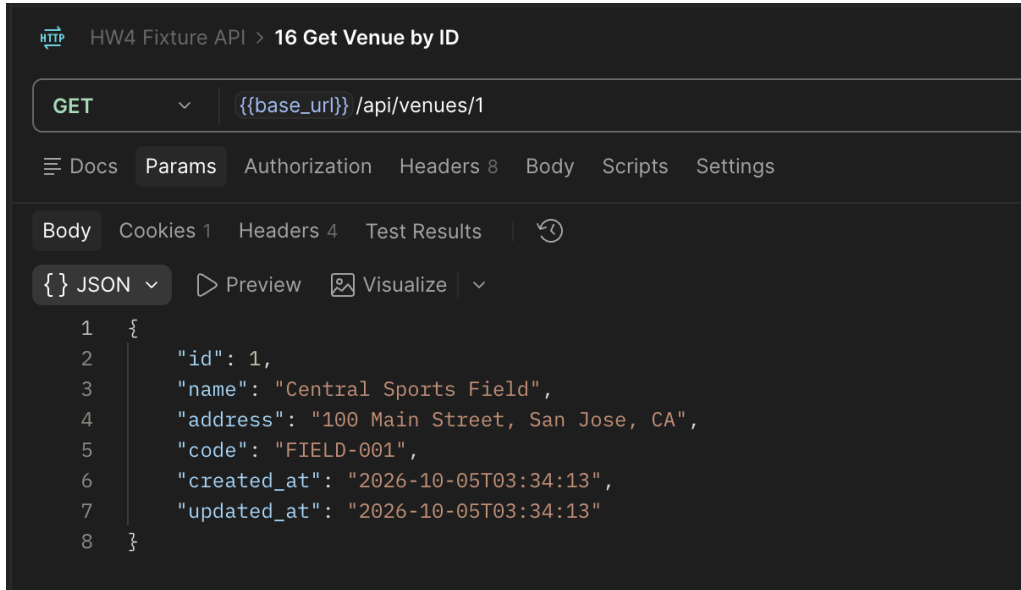
```
@app.get("/api/venues", response_model=list[VenueOut])
def list_venues(
    page: int = Query(default=1, ge=1),
    page_size: int = Query(default=10, ge=1, le=200),
    db: DbSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    offset = (page - 1) * page_size # calculates how many records to skip for the current page

    venues = (
        db.query(Venue)
        .order_by(Venue.id)
        .offset(offset)
        .limit(page_size)
        .all()
    )

    return [
        venue_to_dict(venue)
        for venue in venues
    ]
```

Ananya Yallapragada
019142383

3. Get venue by id



The screenshot shows a REST client interface for the 'HW4 Fixture API'. The selected endpoint is '16 Get Venue by ID' with a GET method. The URL is '{{base_url}} /api/venues/1'. The response body is displayed in JSON format, showing details for a venue with ID 1.

```
1 {
2   "id": 1,
3   "name": "Central Sports Field",
4   "address": "100 Main Street, San Jose, CA",
5   "code": "FIELD-001",
6   "created_at": "2026-10-05T03:34:13",
7   "updated_at": "2026-10-05T03:34:13"
8 }
```

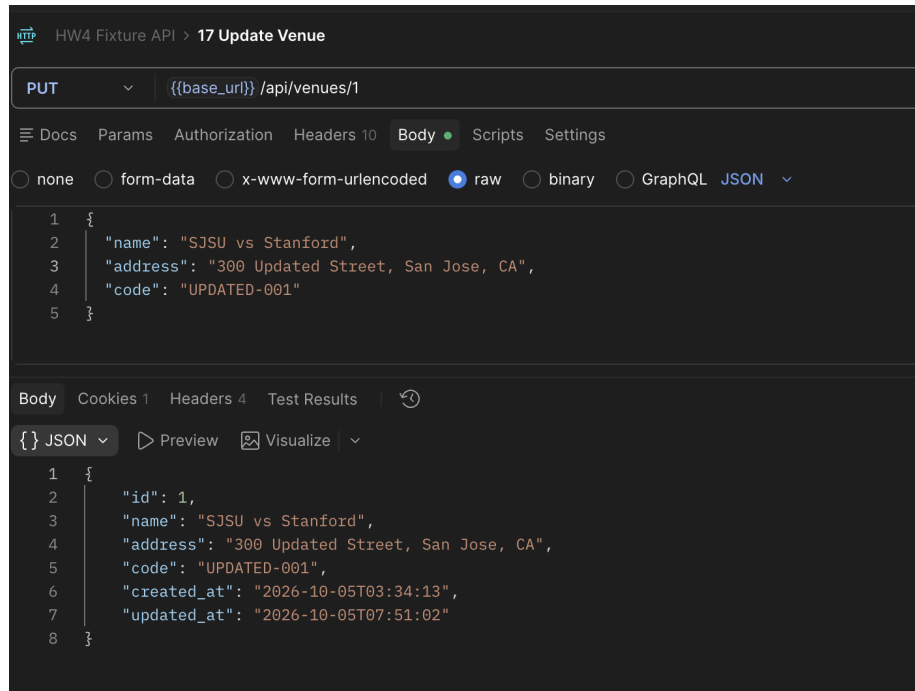
```
@app.get("/api/venues/{venue_id}", response_model=VenueOut)
def get_venue(
    venue_id: int,
    db: DbSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    venue = db.get(Venue, venue_id)

    if not venue:
        raise HTTPException(
            status_code=404,
            detail="Venue not found",
        )

    return venue_to_dict(venue)
```

Ananya Yallapragada
019142383

4. Update Venue



The screenshot shows a REST client interface for a "HW4 Fixture API". The selected endpoint is "17 Update Venue" with a PUT method. The request body is raw JSON:

```
{
  "name": "SJSU vs Stanford",
  "address": "300 Updated Street, San Jose, CA",
  "code": "UPDATED-001"
}
```

. The response body is also raw JSON:

```
{
  "id": 1,
  "name": "SJSU vs Stanford",
  "address": "300 Updated Street, San Jose, CA",
  "code": "UPDATED-001",
  "created_at": "2026-10-05T03:34:13",
  "updated_at": "2026-10-05T07:51:02"
}
```

```
@app.put("/api/venues/{venue_id}", response_model=VenueOut)
def update_venue(
    venue_id: int,
    request_data: VenueUpdate,
    db: DbSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    venue = db.get(Venue, venue_id)

    if not venue:
        raise HTTPException(
            status_code=404,
            detail="Venue not found",
        )

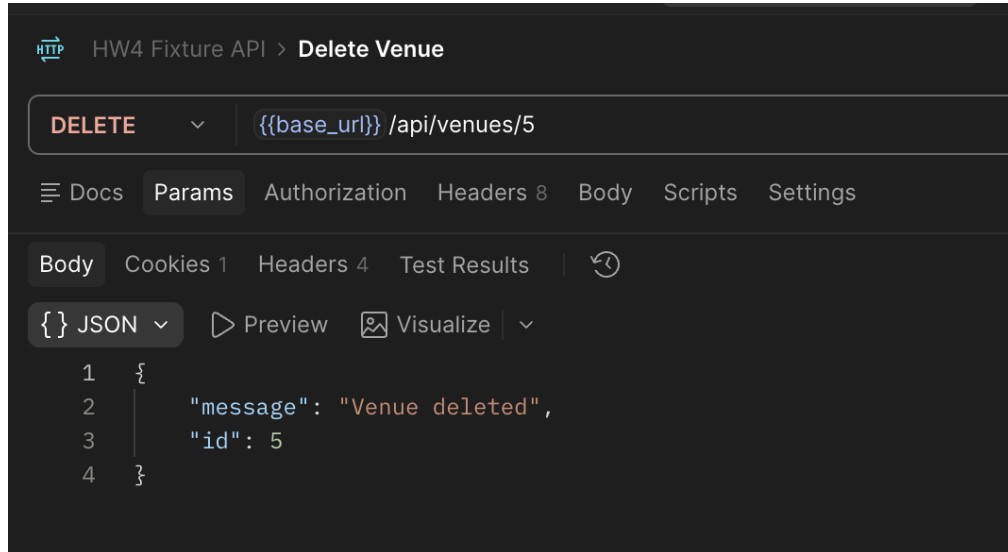
    venue.name = request_data.name
    venue.address = request_data.address
    venue.code = request_data.code

    try:
        db.commit()
        db.refresh(venue)
    except IntegrityError:
        db.rollback()
        raise HTTPException(
            status_code=409,
            detail="Venue code already exists",
        )

    return venue_to_dict(venue)
```

Ananya Yallapragada
019142383

5. Delete



The screenshot shows a REST client interface for a "HW4 Fixture API". The method is set to "DELETE" and the URL is "{{base_url}} /api/venues/5". The response is displayed in JSON format:

```
{
  "message": "Venue deleted",
  "id": 5
}
```

```
@app.delete("/api/venues/{venue_id}")
def delete_venue(
    venue_id: int,
    db: DbSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    venue = db.get(Venue, venue_id)

    if not venue:
        raise HTTPException(
            status_code=404,
            detail="Venue not found",
        )

    fixture_exists = (
        db.query(Fixture)
        .filter(Fixture.venue_id == venue_id)
        .first()
    )

    if fixture_exists:
        raise HTTPException(
            status_code=409,
            detail="Cannot delete a venue that has fixtures",
        )

    db.delete(venue)
    db.commit()

    return {
        "message": "Venue deleted",
        "id": venue_id,
    }
```

Ananya Yallapragada
019142383

Database:

```
mysql> SHOW TABLES;
+-----+
| Tables_in_s2383_rel |
+-----+
| fixture_related_items |
| fixtures              |
| sessions              |
| users                 |
| venues                 |
+-----+
5 rows in set (0.02 sec)
```

```
mysql>
mysql> SELECT id, name, address, code
-> FROM venues;
+----+-----+-----+-----+
| id | name                | address                                     | code      |
+----+-----+-----+-----+
| 1  | SJSU vs Stanford    | 300 Updated Street, San Jose, CA          | UPDATED-001 |
| 2  | foothill college    | 12345 El Monte Rd, Los Altos Hills, CA 94022 | 004       |
| 6  | Postman Sports Field | 200 Test Street, San Jose, CA             | POSTMAN-001 |
+----+-----+-----+-----+
3 rows in set (0.00 sec)

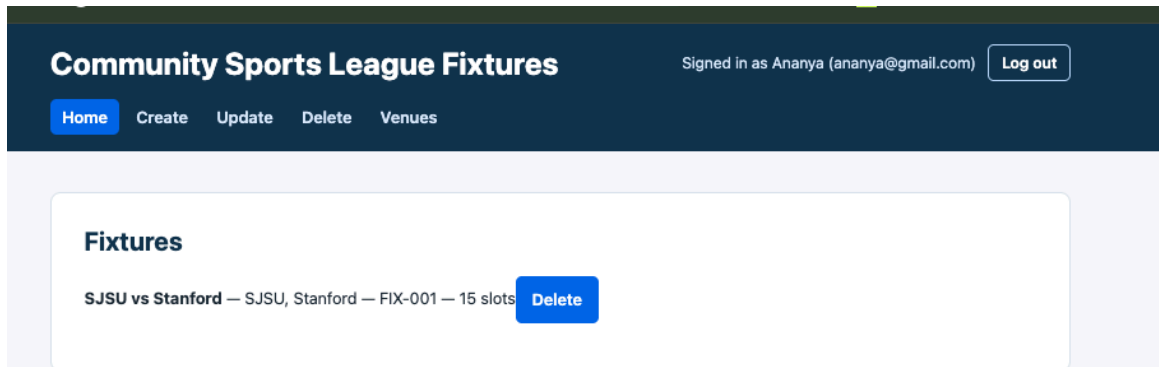
mysql>
mysql> SELECT id, fixture_name, teams, fixture_code,
->         available_slots, venue_id
-> FROM fixtures;
+----+-----+-----+-----+-----+-----+
| id | fixture_name      | teams                | fixture_code | available_slots | venue_id |
+----+-----+-----+-----+-----+-----+
| 1  | SJSU vs Stanford | SJSU, Stanford      | FIX-001     | 15              | 1        |
| 3  | Round 2          | Panthers vs Tigers  | FIX-002     | 12              | 2        |
+----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Ananya Yallapragada
019142383

III. Redux Client

Fixtures

1. Home



```
export default function Home({ user }) {  
  const dispatch = useDispatch();  
  
  const {  
    items: fixtures,  
    loading,  
    error,  
  } = useSelector((state) => state.fixtures);  
  
  useEffect(() => {  
    if (user) {  
      dispatch(fetchFixtures());  
    }  
  }, [user, dispatch]);  
}
```

2. Create

The screenshot shows the 'Create Fixture' form in the 'Community Sports League Fixtures' application. The header and navigation bar are the same as in the previous screenshot. The main content area is light gray and contains a section titled 'Create Fixture'. Below this title is a form with the following fields: 'Fixture name' (text input with 'Round 2'), 'Team one' (text input with 'Panthers'), 'Team two' (text input with 'Tigers'), 'Fixture code' (text input with 'FIX-002'), 'Available slots' (text input with '8'), and 'Venue' (dropdown menu with 'Central Sports Field (FIELD-001)'). A blue 'Create fixture' button is at the bottom of the form.

Ananya Yallapragada
019142383

```
async function handleSubmit(event) {  
  event.preventDefault();  
  
  setError("");  
  setIsSubmitting(true);  
  
  try {  
    await dispatch(  
      createFixture({  
        fixture_name: fixtureName,  
        teams: `${teamOne} vs ${teamTwo}`,  
        fixture_code: fixtureCode,  
        available_slots: Number(availableSlots),  
        venue_id: Number(venueId),  
      })  
    ).unwrap();  
  
    navigate("/");  
  } catch (requestError) {  
    setError(requestError);  
  } finally {  
    setIsSubmitting(false);  
  }  
}
```

3. Update

localhost:5173/update

Model Editing cohort NASA EnsembleCG ChatGPT Home - The Data... How to Find and S... (8) How to be an e... Career Journey | U... Jobs | Handshake Online

Community Sports League Fixtures Signed in as Ananya (ananya@gmail.com) Log out

Home Create **Update** Delete

Update Fixture

Select fixture
Round 2

Fixture name
Round 2

Team one
Panthers

Team two
Tigers

Fixture code
FIX-002

Available slots
12

Venue
Central Sports Field (FIELD-001)

Update fixture

Ananya Yallapragada
019142383

```
async function handleSubmit(event) {
  event.preventDefault();

  setError("");
  setIsSubmitting(true);

  try {
    await dispatch(
      updateFixture({
        id: selectedFixtureId,
        data: {
          fixture_name: fixtureName,
          teams: `${teamOne} vs ${teamTwo}`,
          fixture_code: fixtureCode,
          available_slots: Number(availableSlots),
          venue_id: Number(venueId),
        },
      })
    ).unwrap();

    navigate("/");
  } catch (requestError) {
    setError(requestError);
  } finally {
    setIsSubmitting(false);
  }
}
```

4. Delete

Community Sports League Fixtures Signed in as Ananya (ananya@gmail.com) [Log out](#)

[Home](#) [Create](#) [Update](#) [Delete](#)

Delete Fixture

Select fixture

Round 2 — Panthers vs Tigers

[Delete fixture](#)

localhost:5173 says

Delete "Round 2"?

[Cancel](#) [OK](#)

Delete Fixture

Select fixture

Round 2 — Panthers vs Tigers

[Delete fixture](#)

Ananya Yallapragada
019142383

```
async function handleSubmit(event) {
  event.preventDefault();

  if (!selectedFixtureId) {
    setError("Select a fixture to delete.");
    return;
  }

  const selectedFixture = fixtures.find(
    (fixture) => fixture.id === Number(selectedFixtureId)
  );

  const shouldDelete = window.confirm(
    `Delete "${selectedFixture.fixture_name}"?`
  );

  if (!shouldDelete) {
    return;
  }

  setError("");
  setIsSubmitting(true);

  try {
    await dispatch(deleteFixture(Number(selectedFixtureId))).unwrap();
    navigate("/");
  } catch (requestError) {
    setError(requestError);
  } finally {
    setIsSubmitting(false);
  }
}
```

Part 2

Part A - TheMealDB Tutorial Server

1. search_meals_by_name

search_meals_by_name

Search TheMealDB meals by name.

Query *

Arrabiata

Limit

5

Execute Tool

×

Results

```
1 - {
2   "id": "52771",
3   "name": "Spicy Arrabiata Penne",
4   "area": "Italian",
5   "category": "Vegetarian",
6   "thumb": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg"
7 }
```

Structured Output

```
1 - {
2   "result": [
3     {
4       "id": "52771",
5       "name": "Spicy Arrabiata Penne",
6       "area": "Italian",
7       "category": "Vegetarian",
8       "thumb": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg"
9     }
10  ]
11 }
```

```
@mcp.tool()
def search_meals_by_name(
    query: str,
    limit: int = 5,
) -> list[dict]:
    """Search TheMealDB meals by name."""

    if not query.strip():
        raise ValueError("query cannot be empty")

    if not 1 <= limit <= 25:
        raise ValueError("limit must be between 1 and 25")

    try:
        response = httpx.get(
            f"{BASE_URL}/search.php",
            params={"s": query},
            timeout=10.0,
        )
        response.raise_for_status()
        payload = response.json()
    except httpx.HTTPError as error:
        raise RuntimeError(f"TheMealDB request failed: {error}") from error

    meals = payload.get("meals") or []

    return [
        {
            "id": meal["idMeal"],
            "name": meal["strMeal"],
            "area": meal["strArea"],
            "category": meal["strCategory"],
            "thumb": meal["strMealThumb"],
        }
        for meal in meals[:limit]
    ]
```

Ananya Yallapragada
019142383

2. meals_by_ingridient

meals_by_ingradient

Find meals by main ingredient.

Ingredient *

Limit

Execute Tool

Results

```
1- {
2  "id": "52954",
3  "name": "Hot and Sour Soup",
4  "thumb": "https://www.themeadb.com/images/media/meals/1529445893.jpg"
5 }
```

```
1- {
2  "id": "52947",
3  "name": "Ma Po Tofu",
4  "thumb": "https://www.themeadb.com/images/media/meals/1525874812.jpg"
5 }
```

Structured Output

```
1- {
2-  "result": [
3-    {
4      "id": "52954",
5      "name": "Hot and Sour Soup",
6      "thumb": "https://www.themeadb.com/images/media/meals/1529445893.jpg"
7    },
8    {
9      "id": "52947",
10     "name": "Ma Po Tofu",
11     "thumb": "https://www.themeadb.com/images/media/meals/1525874812.jpg"
12   }
13 ]
14 }
```

```
@mcp.tool()
def meals_by_ingradient(
    ingredient: str,
    limit: int = 12,
) -> list[dict]:
    """Find meals by main ingredient."""

    if not ingredient.strip():
        raise ValueError("ingredient cannot be empty")

    if not 1 <= limit <= 25:
        raise ValueError("limit must be between 1 and 25")

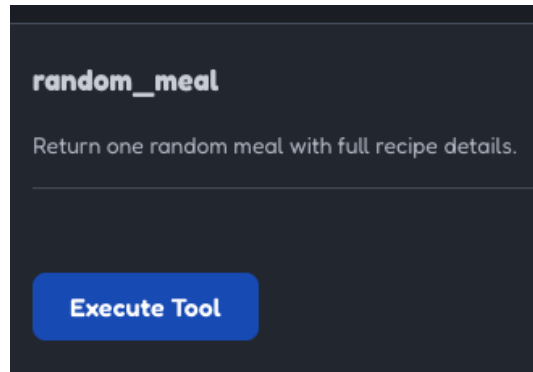
    try:
        response = httpx.get(
            f"{BASE_URL}/filter.php",
            params={"i": ingredient},
            timeout=10.0,
        )
        response.raise_for_status()
        payload = response.json()
    except httpx.HTTPError as error:
        raise RuntimeError(f"TheMealDB request failed: {error}") from error

    meals = payload.get("meals") or []

    return [
        {
            "id": meal["idMeal"],
            "name": meal["strMeal"],
            "thumb": meal["strMealThumb"],
        }
        for meal in meals[:limit]
    ]
```

Ananya Yallapragada
019142383

3. random_meal



```
× Results
1 {
2   "id": "53106",
3   "name": "Warm roast asparagus salad",
4   "category": "Pork",
5   "area": "Australian",
6   "instructions": "step 1\r\nPreheat the oven to 200C/Gas 6/fan 180C. Snap off the woody ends of the asparagus and discard. Arrange
7   "image": "https://www.themealdb.com/images/media/meals/ktsws11761998344.jpg",
8   "source": "https://www.bbcgoodfood.com/recipes/warm-roast-asparagus-salad",
9   "youtube": "https://www.youtube.com/watch?v=pR_ZfbTxwms",
10  "ingredients": [
11    {
12      "name": "Asparagus",
13      "measure": "500g"
14    },
15    {
16      "name": "Red Wine Vinegar",
17      "measure": "2 tbsp"
18    },
19    {
20      "name": "Tomato",
21      "measure": "4"
22    },
23    {
24      "name": "Extra Virgin Olive Oil",
25      "measure": "1 tbsp"
26    },
27    {
28      "name": "Streaky Bacon",
29      "measure": "12"
30    },
31    {
32      "name": "Clear Honey",
33      "measure": "1 teaspoon"
34    },
35    {
36      "name": "Jersey Royal Potatoes",
37      "measure": "16"
38    }
39  ]
40 }
```

```
@mcp.tool()
def random_meal() -> dict:
    """Return one random meal with full recipe details."""
    try:
        response = httpx.get(
            f"{BASE_URL}/random.php",
            timeout=10.0,
        )
        response.raise_for_status()
        payload = response.json()
    except httpx.HTTPError as error:
        raise RuntimeError(f"TheMealDB request failed: {error}") from error

    meals = payload.get("meals") or []

    if not meals:
        raise RuntimeError("TheMealDB returned no random meal")

    return format_meal_details(meals[0])
```

Ananya Yallapragada
019142383

4. meal_details

meal_details

Return full recipe details for one meal ID.

Id *

1

"52772"

```
× Results
1- {
2   "id": "52772",
3   "name": "Teriyaki Chicken Casserole",
4   "category": "Chicken",
5   "area": "Japanese",
6   "instructions": "Preheat oven to 350° F. Spray a 9x13-inch baking pan with non-stick spray.\r\n\r\nCombine soy sauce, ½ cup water, bro
7   "image": "https://www.themealdb.com/images/media/meals/wvpsxx1468256321.jpg",
8   "source": null,
9   "youtube": "https://www.youtube.com/watch?v=4aZr5hZXP_s",
10  "ingredients": [
11    {
12      "name": "soy sauce",
13      "measure": "3/4 cup"
14    },
15    {
16      "name": "water",
17      "measure": "1/2 cup"
18    },
19    {
20      "name": "brown sugar",
21      "measure": "1/4 cup"
22    },
23    {
24      "name": "ground ginger",
25      "measure": "1/2 teaspoon"
26    },
27    {
28      "name": "minced garlic",
29      "measure": "1/2 teaspoon"
30    },
31    {
32      "name": "cornstarch",
33      "measure": "4 Tablespoons"
34    },
35    {
36      "name": "chicken breasts",
37      "measure": "2"
38    },
39  ],
40 }
```

```
@mcp.tool()
def meal_details(id: str | int) -> dict:
    """Return full recipe details for one meal ID."""

    meal_id = str(id).strip()

    if not meal_id:
        raise ValueError("id cannot be empty")

    try:
        response = httpx.get(
            f"{BASE_URL}/lookup.php",
            params={"i": meal_id},
            timeout=10.0,
        )
        response.raise_for_status()
        payload = response.json()
    except httpx.HTTPError as error:
        raise RuntimeError(f"TheMealDB request failed: {error}") from error

    meals = payload.get("meals") or []

    if not meals:
        raise ValueError(f"No meal found for ID {meal_id}")

    return format_meal_details(meals[0])
```

Part B - Domain MCP Server

1. search_fixtures

search_fixtures

Search fixtures by name, teams, or fixture code.

Query *

SJSU

Limit

10

Execute Tool

```
× Results
1 {
2   "ok": true,
3   "data": [
4     {
5       "id": 1,
6       "fixture_name": "SJSU vs Stanford",
7       "teams": "SJSU, Stanford",
8       "fixture_code": "FIX-001",
9       "available_slots": 15,
10      "venue_id": 1,
11      "created_at": "2026-10-05T03:41:07",
12      "updated_at": "2026-10-05T03:50:02",
13      "related_items": []
14    }
15  ],
16  "error": null
17 }
```

search_fixtures

Search fixtures by name, teams, or fixture code.

Query *

SJSU

Limit

0

Execute Tool

```
× Results
1 {
2   "ok": false,
3   "data": null,
4   "error": "limit must be between 1 and 25."
5 }
```

```
@mcp.tool()
async def search_fixtures(query: str, limit: int = 10):
    """Search fixtures by name, teams, or fixture code."""
    query = query.strip().lower()

    if not query:
        return failure("query must not be empty.")

    if limit < 1 or limit > 25:
        return failure("limit must be between 1 and 25.")

    payload, error = await get_json(
        "/fixtures",
        params={"page": 1, "page_size": 100},
    )

    if error:
        return failure(error)

    fixtures = payload if isinstance(payload, list) else []

    matches = []

    for fixture in fixtures:
        searchable_text = (
            f"{fixture.get('fixture_name', '')} "
            f"{fixture.get('teams', '')} "
            f"{fixture.get('fixture_code', '')}"
        ).lower()

        if query in searchable_text:
            matches.append(fixture)

    return success(matches[:limit])
```

Ananya Yallapragada
019142383

2. fixture_details

fixture_details

Return one fixture by ID.

Fixture Id *

3

Execute Tool

× **Results**

```
1 {  
2   "ok": true,  
3   "data": {  
4     "id": 3,  
5     "fixture_name": "Round 2",  
6     "teams": "Panthers vs Tigers",  
7     "fixture_code": "FIX-002",  
8     "available_slots": 12,  
9     "venue_id": 2,  
10    "created_at": "2026-10-05T08:01:09",  
11    "updated_at": "2026-10-05T08:01:25",  
12    "related_items": []  
13  },  
14  "error": null  
15 }
```

fixture_details

Return one fixture by ID.

Fixture Id *

10

Execute Tool

× **Results**

```
1 {  
2   "ok": false,  
3   "data": null,  
4   "error": "Resource not found."  
5 }
```

```
@mcp.tool()  
async def fixture_details(fixture_id: int):  
    """Return one fixture by ID."""  
    if fixture_id < 1:  
        return failure("fixture_id must be a positive integer.")  
  
    payload, error = await get_json(f"/fixtures/{fixture_id}")  
  
    if error:  
        return failure(error)  
  
    return success(payload)
```

Ananya Yallapragada
019142383

3. venue_fixture_summary

venue_fixture_summary

Return an aggregate summary of fixtures at a venue.

Venue Id *

1

Execute Tool

× Results

```
1 {
2   "ok": true,
3   "data": {
4     "venue_id": 1,
5     "fixture_count": 1,
6     "total_available_slots": 15,
7     "fixture_names": [
8       "SJSU vs Stanford"
9     ]
10  },
11  "error": null
12 }
```

venue_fixture_summary

Return an aggregate summary of fixtures at a venue.

Venue Id *

9

Execute Tool

× Results

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "Resource not found."
5 }
```

```
@mcp.tool()
async def venue_fixture_summary(venue_id: int):
    """Return an aggregate summary of fixtures at a venue."""
    if venue_id < 1:
        return failure("venue_id must be a positive integer.")

    payload, error = await get_json(
        f"/venues/{venue_id}/fixtures",
        params={"page": 1, "page_size": 100},
    )

    if error:
        return failure(error)

    fixtures = get_items(payload)

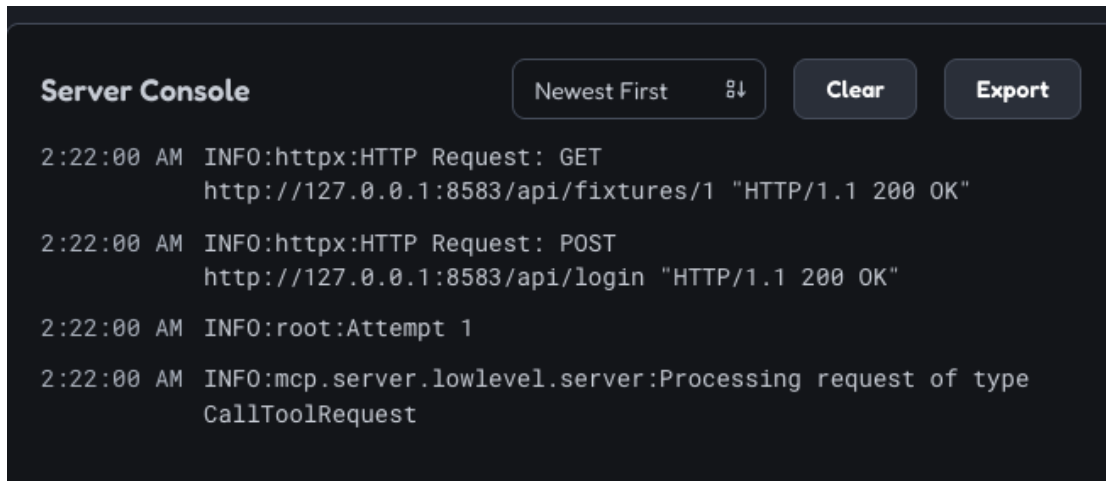
    total_available_slots = sum(
        int(fixture.get("available_slots", 0))
        for fixture in fixtures
    )

    summary = {
        "venue_id": venue_id,
        "fixture_count": len(fixtures),
        "total_available_slots": total_available_slots,
        "fixture_names": [
            fixture.get("fixture_name")
            for fixture in fixtures
        ],
    }

    return success(summary)
```

Part 3 - Tool Contracts Under Stress

1. Success on the first attempt - FAILURE_RATE = 0.0



The screenshot shows a 'Server Console' window with a dark background. At the top, there are three buttons: 'Newest First' with a sort icon, 'Clear', and 'Export'. Below the buttons, the console displays the following log entries:

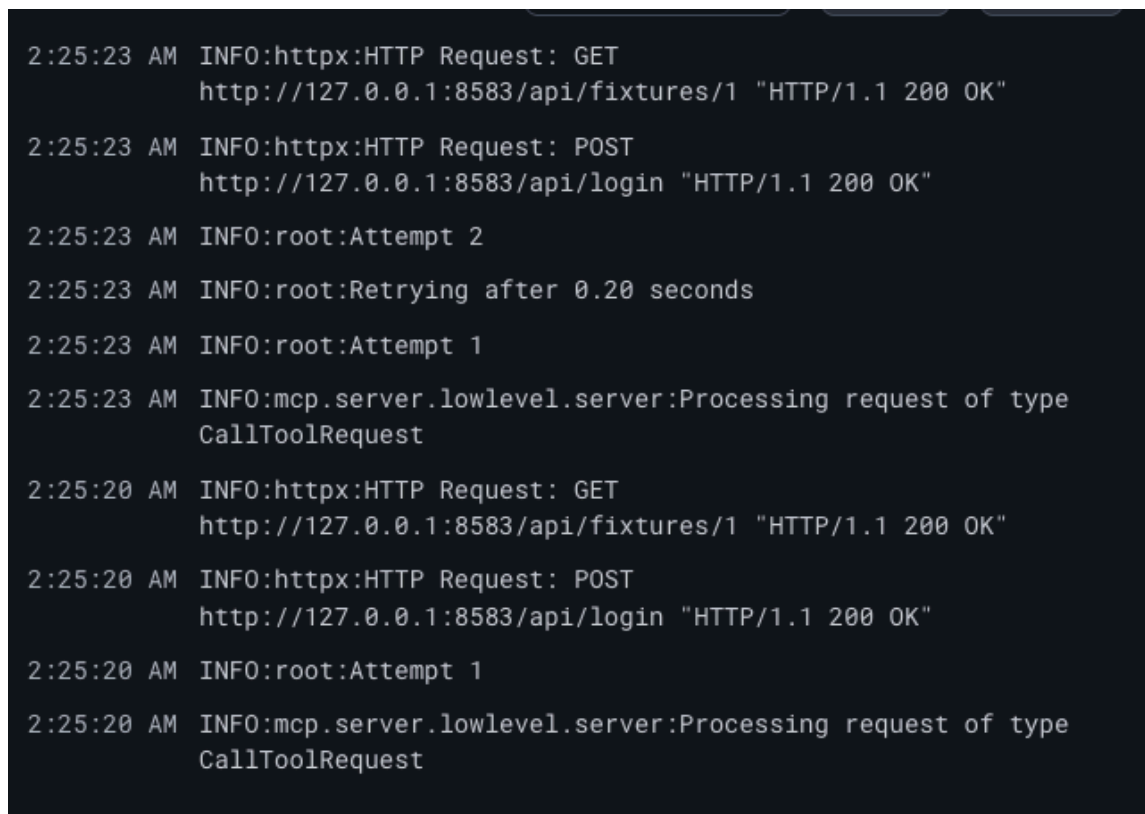
```
2:22:00 AM INFO:httpx:HTTP Request: GET
http://127.0.0.1:8583/api/fixtures/1 "HTTP/1.1 200 OK"

2:22:00 AM INFO:httpx:HTTP Request: POST
http://127.0.0.1:8583/api/login "HTTP/1.1 200 OK"

2:22:00 AM INFO:root:Attempt 1

2:22:00 AM INFO:mcp.server.lowlevel.server:Processing request of type
CallToolRequest
```

2. Failure on the first attempt followed by success after a retry - FAILURE_RATE = 0.2



The screenshot shows a 'Server Console' window with a dark background. At the top, there are three buttons: 'Newest First' with a sort icon, 'Clear', and 'Export'. Below the buttons, the console displays the following log entries:

```
2:25:23 AM INFO:httpx:HTTP Request: GET
http://127.0.0.1:8583/api/fixtures/1 "HTTP/1.1 200 OK"

2:25:23 AM INFO:httpx:HTTP Request: POST
http://127.0.0.1:8583/api/login "HTTP/1.1 200 OK"

2:25:23 AM INFO:root:Attempt 2

2:25:23 AM INFO:root:Retrying after 0.20 seconds

2:25:23 AM INFO:root:Attempt 1

2:25:23 AM INFO:mcp.server.lowlevel.server:Processing request of type
CallToolRequest

2:25:20 AM INFO:httpx:HTTP Request: GET
http://127.0.0.1:8583/api/fixtures/1 "HTTP/1.1 200 OK"

2:25:20 AM INFO:httpx:HTTP Request: POST
http://127.0.0.1:8583/api/login "HTTP/1.1 200 OK"

2:25:20 AM INFO:root:Attempt 1

2:25:20 AM INFO:mcp.server.lowlevel.server:Processing request of type
CallToolRequest
```

Ananya Yallapragada
019142383

3. Failure after all allowed retries, returning a clean error result instead of crashing - `FAILURE_RATE = 0.5`

```
2:27:26 AM ERROR:root:Request failed: Simulated failure
2:27:26 AM INFO:root:Attempt 3
2:27:26 AM INFO:root:Retrying after 0.40 seconds
2:27:26 AM INFO:root:Attempt 2
2:27:25 AM INFO:root:Retrying after 0.20 seconds
2:27:25 AM INFO:root:Attempt 1
2:27:25 AM INFO:mcp.server.lowlevel.server:Processing request of type
CallToolRequest
```

Snippets for the above:

```
async def run_with_retries(operation):
    for attempt in range(MAX_RETRIES + 1):
        logging.info("Attempt %d", attempt + 1)

        try:
            if random_generator.random() < FAILURE_RATE:
                raise httpx.RequestError("Simulated failure")

            return await operation()

        except httpx.RequestError:
            if attempt == MAX_RETRIES:
                raise

            delay = BACKOFF_SECONDS * (2 ** attempt)
            logging.info("Retrying after %.2f seconds", delay)
            await asyncio.sleep(delay)
```

```
for rate in RATES:
    domain_server.FAILURE_RATE = rate
    domain_server.random_generator = random.Random(SEED)

    records = []
    latencies = []

    for call_number in range(1, CALLS_PER_RATE + 1):
        start = time.perf_counter()

        result = await domain_server.fixture_details(1)

        elapsed_ms = (time.perf_counter() - start) * 1000
        latencies.append(elapsed_ms)

        records.append(
            {
                "seed": SEED,
                "failure_rate": rate,
                "call_number": call_number,
                "ok": result.get("ok") is True,
                "latency_ms": round(elapsed_ms, 3),
                "result": result,
            }
        )
    )
```

Part 4 - One Safe Tool Entry Point and Offline Tests

```
0001219:1cc90505 main -> main
● (.venv) (base) annie@mac data260-2383 % python code/mcp/test_tools.py
PASS: search valid input
PASS: search invalid limit
PASS: fixture details valid input
PASS: fixture details missing fixture
PASS: venue summary valid input
PASS: venue summary missing venue

6/6 tests passed
```

```
async def execute_tool(name: str, inputs: dict):
    """Safely execute one of the three domain tools."""
    try:
        if name == "search_fixtures":
            result = await search_fixtures(**inputs)

        elif name == "fixture_details":
            result = await fixture_details(**inputs)

        elif name == "venue_fixture_summary":
            result = await venue_fixture_summary(**inputs)

        else:
            result = failure(f"Unknown tool: {name}")

        return json.dumps(result)

    except Exception as error:
        logging.error("execute_tool failed: %s", error)

        return json.dumps([
            failure("Tool execution failed.")
        ])
```

```
async def fake_get_json(path, params=None):
    if path == "/fixtures":
        return [FIXTURE], None

    if path == "/fixtures/1":
        return FIXTURE, None

    if path == "/venues/1/fixtures":
        return [FIXTURE], None

    if path in ["/fixtures/10", "/venues/9/fixtures"]:
        return None, "Resource not found."

    return None, "Unexpected test path."

domain_server.get_json = fake_get_json
domain_server.FAILURE_RATE = 0.0

async def run_test(name, tool_name, inputs, expected_ok):
    raw_result = await domain_server.execute_tool(
        tool_name,
        inputs,
    )

    result = json.loads(raw_result)

    assert result["ok"] is expected_ok

    print(f"PASS: {name}")
```

Ananya Yallapragada
019142383

Part 5 - Agent Loop, Safety Rule, and Reflection

1. safety allowed/blocked

```
INFO:root:Attempt 1
INFO:httpx:HTTP Request: POST http://127.0.0.1:8583/api/login "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: GET http://127.0.0.1:8583/api/fixtures?page=1&page_size=100 "HTTP/1.1 200 OK"
ALLOWED CALL
{
  "ok": true,
  "data": [
    {
      "id": 1,
      "fixture_name": "SJSU vs Stanford",
      "teams": "SJSU, Stanford",
      "fixture_code": "FIX-001",
      "available_slots": 15,
      "venue_id": 1,
      "created_at": "2026-10-05T03:41:07",
      "updated_at": "2026-10-05T03:50:02",
      "related_items": []
    }
  ],
  "error": null
}

BLOCKED CALL
{
  "ok": false,
  "data": null,
  "error": "Destructive commands are not allowed."
}
```

```
if name == "search_fixtures":
    query = str(inputs.get("query", "")).lower()

    blocked_terms = [
        "delete",
        "drop",
        "truncate",
        "update",
        "insert",
    ]

    if any(term in query for term in blocked_terms):
        return json.dumps(
            failure(
                "Destructive commands are not allowed."
            )
        )
```

Ananya Yallapragada

019142383

2. Agent

```
(.venv) (base) annie@mac data260-2383 % python code/mcp/agent.py "What fixtures are scheduled at venue 1?"
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
INFO:root:Attempt 1
INFO:httpx:HTTP Request: POST http://127.0.0.1:8583/api/login "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: GET http://127.0.0.1:8583/api/venues/1/fixtures?page=1&page_size=100 "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
{
  "user_input": "What fixtures are scheduled at venue 1?",
  "steps": [
    {
      "step": 1,
      "model_output": "{\"action\": \"tool\", \"tool\": \"venue_fixture_summary\", \"inputs\": {\"venue_id\": 1}}",
      "tool": "venue_fixture_summary",
      "inputs": {
        "venue_id": 1
      },
      "result": {
        "ok": true,
        "data": {
          "venue_id": 1,
          "fixture_count": 1,
          "total_available_slots": 15,
          "fixture_names": [
            "SJSU vs Stanford"
          ]
        },
        "error": null
      }
    },
    {
      "step": 2,
      "model_output": "{\"action\": \"final\", \"answer\": \"At venue 1, the following fixture is scheduled: \\n- SJSU vs Stanford\\n\\nTotal available slots: 15\"}",
      "stop_reason": "normal_completion"
    }
  ],
  "final_answer": "At venue 1, the following fixture is scheduled: \\n- SJSU vs Stanford\\n\\nTotal available slots: 15",
  "stop_reason": "normal_completion"
}
```

```
for step_number in range(1, max_steps + 1):
    model_output = await model_call(messages)

    step_record = {
        "step": step_number,
        "model_output": model_output,
    }

    try:
        decision = parse_json(model_output)
    except json.JSONDecodeError:
        stop_reason = "invalid_model_output"
        step_record["error"] = "Model did not return valid JSON."
        steps.append(step_record)
        break

    if decision.get("action") == "final":
        final_answer = decision.get("answer", "")
        stop_reason = "normal_completion"
        step_record["stop_reason"] = stop_reason
        steps.append(step_record)
        break

    if decision.get("action") != "tool":
        stop_reason = "invalid_action"
        step_record["error"] = "Unknown model action."
        steps.append(step_record)
        break

    tool_name = decision.get("tool")
    inputs = decision.get("inputs", {})

    tool_result = await domain_server.execute_tool(
        tool_name,
        inputs,
    )
```

3. Tests

```
• (.venv) (base) annie@mac data260-2383 % python code/mcp/test_tools.py
PASS: search valid input
PASS: search invalid limit
PASS: fixture details valid input
PASS: fixture details missing fixture
PASS: venue summary valid input
PASS: venue summary missing venue
PASS: safety rule blocks destructive query
PASS: MockModel stops at max_steps

8/8 tests passed
```

```
assert blocked_result["ok"] is False
print("PASS: safety rule blocks destructive query")
passed += 1
```

```
assert agent_result["stop_reason"] == "max_steps"
assert len(agent_result["steps"]) == 2
```

V. Reflection:

I reviewed the run for the request - 'What fixtures are scheduled at venue 1?' The agent began with the request and sent it to the local 'qwen3:8b' model through Ollama. The model selected the 'venue_fixture_summary' tool and supplied 'venue_id: 1' as the input.

The agent passed this tool call to 'execute_tool'. The tool contacted the FastAPI backend, authenticated successfully, and retrieved the fixtures associated with venue 1. The returned envelope indicated success and contained one fixture, 'SJSU vs Stanford,' with 15 available slots in total. The result was recorded in the agent run log.

The tool result was then added to the conversation history and sent back to the local model. On the second step, the model interpreted the tool result and produced a final answer stating that venue 1 had one scheduled fixture and 15 available slots. Because the model returned a final answer, the loop stopped with the reason 'normal_completion'.

The run used one tool call and completed in two steps, below the maximum step limit of three. The safety rule was not triggered because the request was a normal read-only domain request. This run demonstrates the intended agent flow: interpret the request, select an appropriate tool, execute the tool safely, use the returned data, and stop normally after producing a final answer.